UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

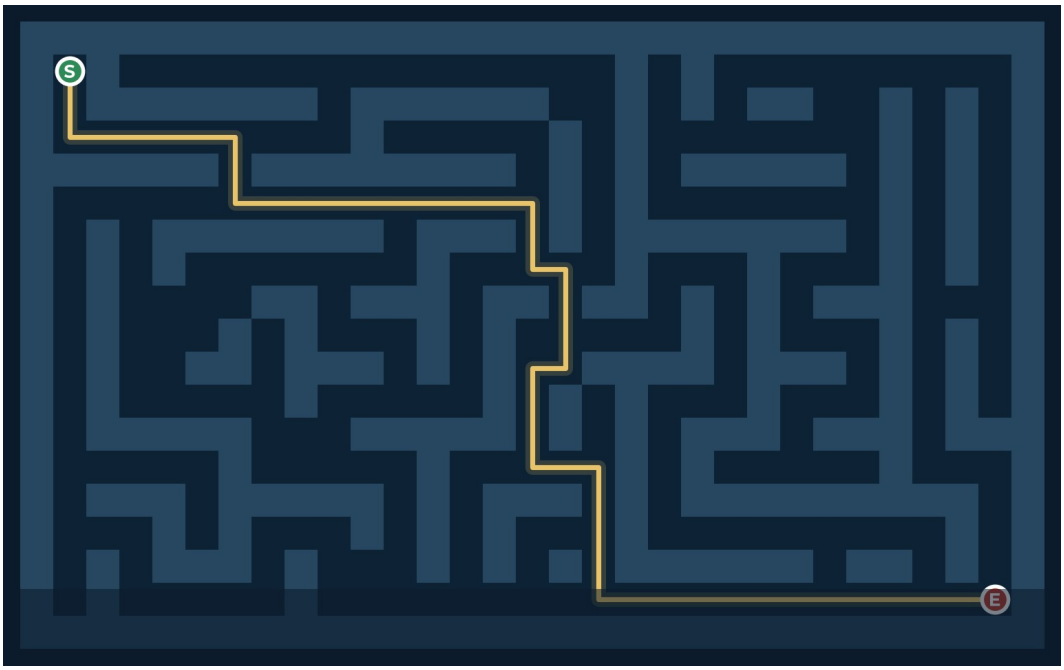
FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL

BÚSQUEDA CIEGA

BFS y DFS para resolver un laberinto

ACTIVIDAD 2.4

ASIGNATURA	Fundamentos de Inteligencia Artificial
ESTUDIANTE	Julissa Marin Flor
DOCENTE	Javier García Lara
GRUPO	G171
INSTITUCIÓN	Universidad Politécnica de Atlautla
FECHA	25 de octubre de 2026



«Elegir un algoritmo es elegir qué precio estamos dispuestos a pagar: BFS paga memoria para garantizar el camino más corto; DFS paga calidad para explorar con menos nodos.»

Contenido

01 **Introducción y objetivo**
Contexto, enunciado y alcance de la actividad.

02 **Representación del problema**
Del laberinto al grafo: nodos, aristas y complejidad.

03 **Búsqueda en amplitud (BFS)**
Estrategia, pseudocódigo y propiedades.

04 **Búsqueda en profundidad (DFS)**
Estrategia, pseudocódigo y propiedades.

05 **Implementación en Python**
Estructura del proyecto y ejecución sin dependencias.

06 **Resultados experimentales**
Caminos solución, nodos expandidos y evolución de la frontera.

07 **Análisis comparativo**
Tiempo, nodos expandidos y calidad de la solución.

08 **Complejidad e idoneidad**
Coste temporal y espacial; elección según el problema.

09 **Conclusiones**
Hallazgos principales y recomendaciones.

10 **Referencias y declaración de IA**
Fuentes consultadas y uso de herramientas de IA.

A **Anexo A — Código completo**
Listado íntegro de los archivos de la carpeta codigo/.

Cómo leer este documento

Las figuras y tablas están numeradas en orden de aparición. Los resultados numéricos provienen de ejecutar codigo/main.py; el anexo incluye el código completo y cada archivo puede abrirse directamente en Visual Studio Code.

1. Introducción y objetivo

La búsqueda no informada, también llamada búsqueda ciega, resuelve problemas explorando el espacio de estados sin utilizar conocimiento adicional sobre cuál estado es más prometedor. Los dos algoritmos clásicos de esta familia son la búsqueda en amplitud (Breadth-First Search, BFS) y la búsqueda en profundidad (Depth-First Search, DFS). Ambos comparten la representación del problema, pero difieren en el orden en que exploran los nodos y, por tanto, en la calidad de la solución y en el uso de memoria.

En esta actividad se implementan BFS y DFS en Python para resolver un laberinto de 31 × 19 celdas. El laberinto se modela como un grafo no dirigido y no ponderado, y el objetivo es encontrar un camino desde la celda de inicio S hasta la salida E. Además de obtener la solución, se comparan los algoritmos en términos de camino, nodos expandidos, aristas evaluadas, tamaño de la frontera y tiempo de ejecución.

Objetivo general: implementar y comparar BFS y DFS sobre una misma instancia, identificando en qué situaciones conviene cada uno.

- **Representar:** el laberinto como un grafo implícito no ponderado;
- **Implementar:** ambos algoritmos con la biblioteca estándar de Python, sin dependencias externas;
- **Medir:** obtener el camino solución, los nodos expandidos, las aristas evaluadas, la frontera máxima y el tiempo;
- **Analizar:** la complejidad temporal y espacial y emitir una conclusión sobre la idoneidad de cada algoritmo.

> **Un detalle importante**
>
> El código de la carpeta codigo/ no requiere instalar librerías: se ejecuta con Python 3.10 o superior usando solo la biblioteca estándar. Las dependencias de este generador (matplotlib y python-docx) se usan únicamente para elaborar el documento.

2. Representación del problema

2.1. El laberinto

La instancia es un laberinto de 31 columnas por 19 filas: 589 celdas en total, de las cuales 290 son transitables y 299 son muros. El inicio está en la celda (1, 1) (fila, columna) y la meta en (17, 29). La Figura 1 muestra el laberinto completo, con S en verde y E en rojo.



Figura 1. *Laberinto 31 × 19 utilizado en la actividad.*

2.2. Del laberinto al grafo

Para aplicar algoritmos de búsqueda, el laberinto se transforma en un grafo G = (V, E) de la siguiente manera:

- **Nodo:** cada celda transitable (incluidas S y E) es un nodo del grafo;
- **Arista:** dos celdas adyacentes en horizontal o vertical, sin muro entre ellas, están unidas por una arista;
- **Peso:** todas las aristas cuestan lo mismo (1 movimiento), por lo que el grafo es no ponderado y no dirigido.

Con estos datos, el grafo tiene V = 290 nodos y E = 315 aristas. El grado medio es 2E/V ≈ 2.17 y el grado máximo es 4, porque cada celda tiene como mucho cuatro vecinos. El grafo no se almacena en memoria: se genera de forma implícita a partir de la matriz del laberinto mediante la función vecinos(). La Figura 2 ilustra la correspondencia entre celdas y nodos.



Figura 2. *El laberinto como grafo: celdas = nodos, pasos = aristas.*

2.3. Un laberinto con bucles

La instancia se generó deliberadamente con bucles, es decir, con varios caminos posibles entre S y E. Si el laberinto fuera perfecto (un único camino), BFS y DFS encontrarían exactamente la misma ruta y la diferencia de optimalidad quedaría oculta. Al abrir algunos muros se crean rutas alternativas: BFS devuelve la más corta y DFS puede devolver una más larga, que es precisamente lo que se quiere comparar.

> **El problema de búsqueda**
>
> Estado inicial: S = (1, 1). Acciones: arriba, abajo, izquierda y derecha. Modelo de transición: moverse a una celda adyacente sin muro. Test de objetivo: estar en E = (17, 29). Coste: 1 por movimiento.

3. Búsqueda en amplitud (BFS)

3.1. Idea del algoritmo

BFS utiliza una cola FIFO (first in, first out). En cada paso extrae el nodo descubierto hace más tiempo, es decir, el más cercano al inicio, y expande todos sus vecinos. De esta forma recorre el grafo por niveles: primero los nodos a distancia 1, luego los de distancia 2, y así sucesivamente. La exploración avanza como una oleada desde S.

En un grafo no ponderado, la primera vez que se extrae la meta de la cola se ha encontrado el camino con el menor número de aristas. Por eso BFS es óptimo cuando todas las acciones tienen el mismo coste.

3.2. Pseudocódigo

```
BFS(inicio, meta):
    padre <- {inicio: None}
    frontera <- Cola()
    frontera.encolar(inicio)

    mientras frontera no este vacia:
        actual <- frontera.desencolar()
        si actual = meta:
            terminar
        para cada vecino de actual:
            si vecino no esta en padre:
                padre[vecino] <- actual
                frontera.encolar(vecino)

    devolver reconstruir_camino(padre, meta)
```

3.3. Propiedades y complejidad

- **¿Es completo?** Sí, en grafos finitos con conjunto de visitados.
- **¿Es óptimo?** Sí, para grafos no ponderados: devuelve el camino con menos aristas.
- **Tiempo:** O(V + E), porque cada nodo y cada arista se procesan como máximo una vez.
- **Espacio:** O(V), por la cola de frontera y el diccionario de padres.

En esta instancia, BFS expandió 214 nodos, alcanzó una frontera máxima de 13 nodos y encontró un camino de 47 celdas (46 movimientos). Es el camino más corto posible entre S y E.

4. Búsqueda en profundidad (DFS)

4.1. Idea del algoritmo

DFS utiliza una pila LIFO (last in, first out). En cada paso extrae el nodo descubierto más recientemente y continúa por esa rama hasta que no puede avanzar; entonces retrocede (backtracking) y prueba otra alternativa. Su recorrido es profundo y no sigue un orden por niveles.

DFS encuentra algún camino si el grafo es finito y se lleva un registro de nodos visitados, pero no garantiza que ese camino sea el más corto. El resultado depende del orden en que se generan los vecinos: si se prueba primero una rama larga, la solución puede ser muy inferior a la de BFS.

4.2. Pseudocódigo

```
DFS(inicio, meta):
    padre <- {inicio: None}
    frontera <- Pila()
    frontera.apilar(inicio)

    mientras frontera no este vacia:
        actual <- frontera.desapilar()
        si actual = meta:
            terminar
        para cada vecino de actual (en orden inverso):
            si vecino no esta en padre:
                padre[vecino] <- actual
                frontera.apilar(vecino)

    devolver reconstruir_camino(padre, meta)
```

4.3. Propiedades y complejidad

- **¿Es completo?** Sí, en grafos finitos, siempre que se marquen los nodos visitados para evitar ciclos.
- **¿Es óptimo?** No: devuelve el primer camino que encuentra, no necesariamente el más corto.
- **Tiempo:** O(V + E), igual que BFS.
- **Espacio:** O(V) en búsqueda sobre grafos con conjunto de visitados; en búsqueda sobre árboles, O(b·m), donde b es el factor de ramificación y m la profundidad máxima.

En esta instancia, DFS expandió 175 nodos, alcanzó una frontera máxima de 19 nodos y encontró un camino de 85 celdas (84 movimientos), claramente más largo que el de BFS. No es un camino óptimo.

5. Implementación en Python

5.1. Estructura del proyecto

El proyecto está organizado en tres módulos y un archivo de arranque. La separación permite reutilizar la instancia, los algoritmos y la presentación de resultados de forma independiente.

Tabla 1. Estructura de la carpeta codigo/

Archivo	Responsabilidad
laberinto.py	Define el laberinto, el inicio, la meta, la función de vecinos y la impresión en ASCII.
busqueda_ciega.py	Implementa BFS y DFS, mide nodos expandidos, aristas evaluadas, frontera y tiempo.
main.py	Ejecuta ambos algoritmos, muestra las soluciones, compara los resultados y guarda resultados_ejecucion_2.4.txt.
README.md	Instrucciones de ejecución y requisitos.
requirements.txt	Sin dependencias externas (solo informativo).

5.2. Fragmentos clave

La generación de vecinos es el corazón de la representación implícita. Solo se devuelven celdas dentro de los límites y sin muro:

```
def vecinos(pos):
    f, c = pos
    for df, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
        candidato = (f + df, c + dc)
        if es_transitable(candidato):
            yield candidato
```

El bucle central de BFS usa una cola y marca cada nodo al encolarlo. Así se evita procesarlo dos veces y se garantiza que el primer camino encontrado sea el más corto:

```
padre = {inicio: None}
frontera = deque([inicio])

while frontera:
    actual = frontera.popleft()
    if actual == meta:
        break
    for vecino in vecinos(actual):
        aristas_evaluadas += 1
        if vecino not in padre:
            padre[vecino] = actual
            frontera.append(vecino)
```

El bucle de DFS es idéntico salvo que utiliza una lista como pila (pop en el extremo final) y apila los vecinos en orden inverso para explorar primero arriba, luego abajo, izquierda y derecha:

```
frontera = [inicio]

while frontera:
    actual = frontera.pop()
    if actual == meta:
        break
    candidatos = list(vecinos(actual))
    aristas_evaluadas += len(candidatos)
    for vecino in reversed(candidatos):
        if vecino not in padre:
            padre[vecino] = actual
            frontera.append(vecino)
```

5.3. Ejecución sin dependencias

Basta con abrir una terminal dentro de la carpeta codigo/ y ejecutar:

```
python main.py
```

En Visual Studio Code se puede abrir la carpeta codigo/, seleccionar main.py y pulsar el botón Run Python File ( ) o Ctrl + F5. No es necesario crear un entorno virtual ni ejecutar pip install: el código solo importa módulos de la biblioteca estándar (collections, time, dataclasses, pathlib y typing).

> **Salida del programa**
> El programa imprime el laberinto, las dos soluciones en ASCII, la tabla comparativa y la conclusión. Además, guarda el archivo resultados_ejecucion_2.4.txt en la carpeta raíz de la actividad.

6. Resultados experimentales

6.1. Camino solución

Ambos algoritmos encuentran un camino entre S y E, pero no son equivalentes. BFS devuelve una ruta de 47 celdas (46 movimientos), mientras que DFS devuelve una de 85 celdas (84 movimientos). La ruta de DFS es aproximadamente un 81% más larga que la de BFS.

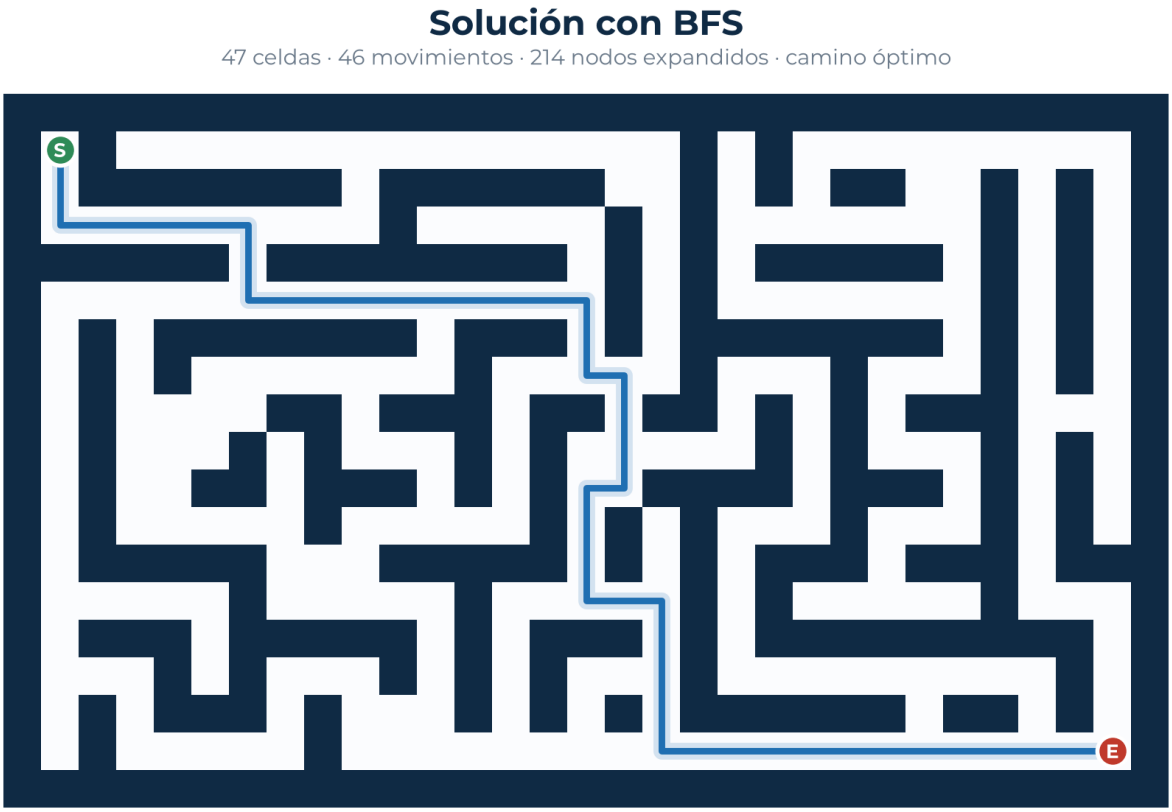


Figura 3. *Camino solución obtenido con BFS (azul).*



Figura 4. *Camino solución obtenido con DFS (naranja).*

La diferencia visual es clara: el camino de BFS avanza de forma directa hacia la meta, mientras que el de DFS rodea y retrocede por ramas que no conducen a la salida. Esto no se debe a un error del algoritmo, sino a su

propia estrategia: DFS se compromete con la primera rama que encuentra y solo cambia cuando esa rama se agota.

6.2. Orden de exploración

Las Figuras 5 y 6 muestran el orden en que se expandió cada nodo. El color claro indica una expansión temprana y el oscuro, una tardía. En BFS se aprecian oleadas concéntricas desde S; en DFS se observa una exploración profunda, con retrocesos cuando una rama no lleva a la meta.

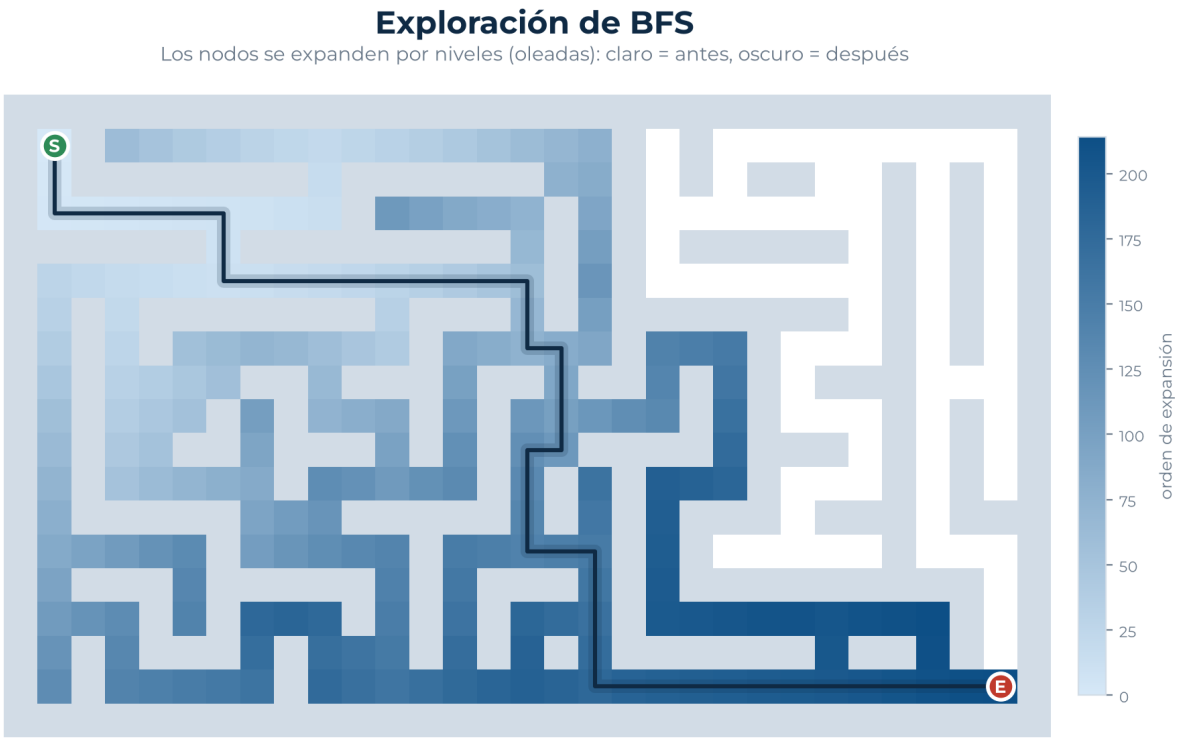


Figura 5. *Orden de expansión de BFS: exploración por niveles.*

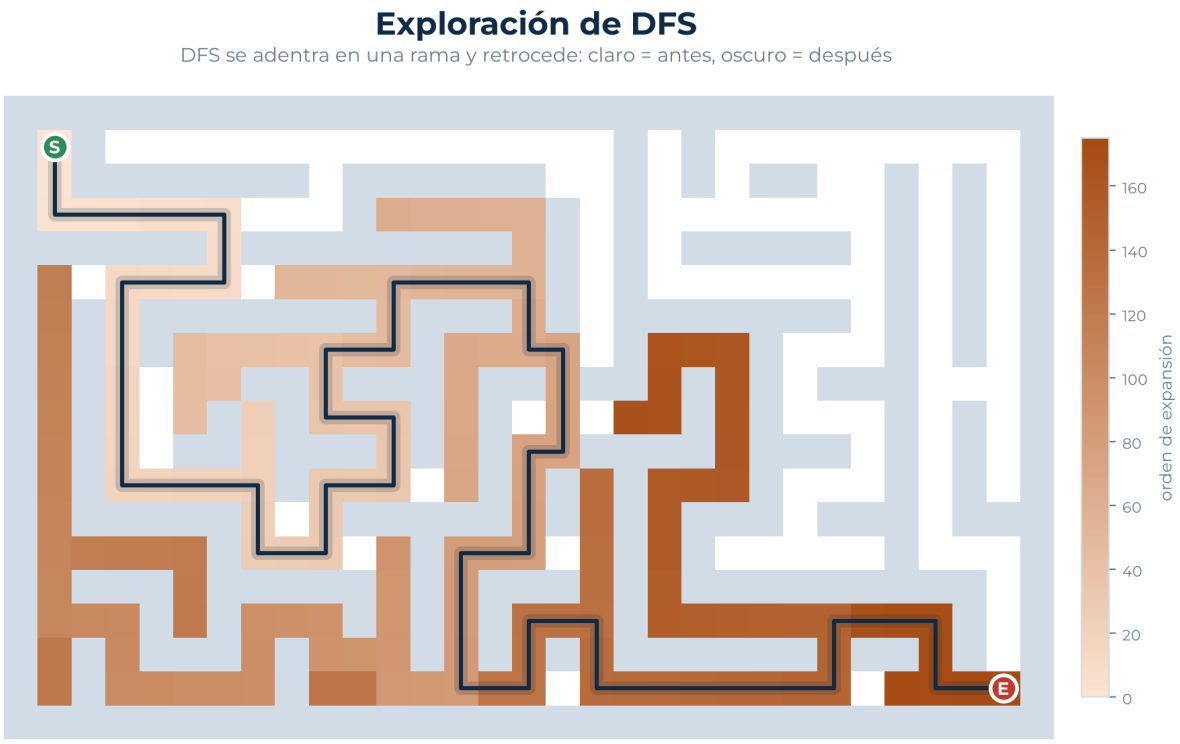


Figura 6. *Orden de expansión de DFS: exploración profunda con retrocesos.*

6.3. Comparación numérica

La Tabla 2 resume las métricas principales. Los tiempos se midieron con 200 ejecuciones por algoritmo (mejor promedio de 5 lotes) para reducir el ruido del sistema operativo.

Tabla 2. Comparación de BFS y DFS en el laberinto 31 × 19.

Métrica	BFS	DFS
¿Encuentra solución?	Sí	Sí
Celdas del camino	47	85
Movimientos (aristas)	46	84
Nodos expandidos	214	175
Aristas evaluadas	467	382
Frontera máxima	13	19
Tiempo medio (ms)	0.720	0.703
¿Camino óptimo?	Sí	No

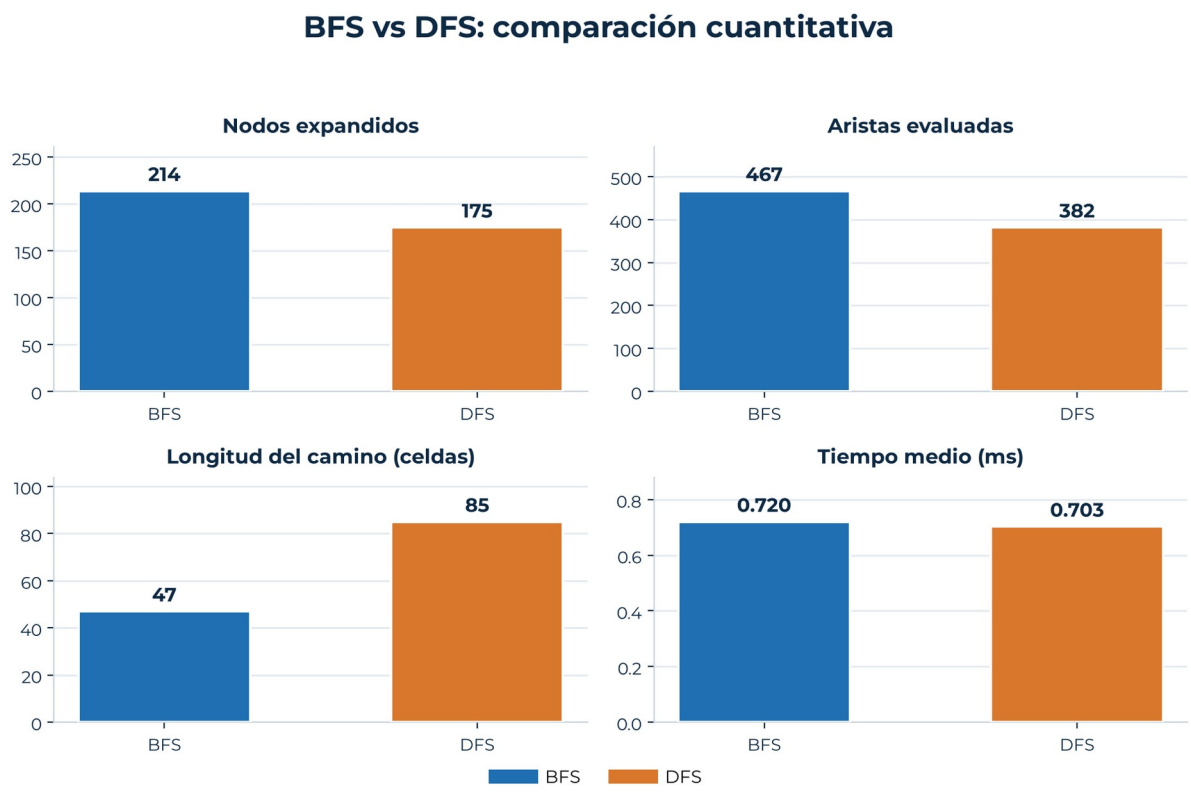


Figura 7. *Comparación cuantitativa de las métricas principales.*

6.4. Evolución de la frontera

La frontera es el conjunto de nodos descubiertos que aún no se han expandido. Su tamaño máximo es una medida directa del consumo de memoria. La Figura 8 muestra cómo cambia a lo largo de la búsqueda: BFS mantiene una frontera relativamente estable, mientras que DFS presenta picos cuando apila todos los vecinos de un nodo antes de seguir por una rama.

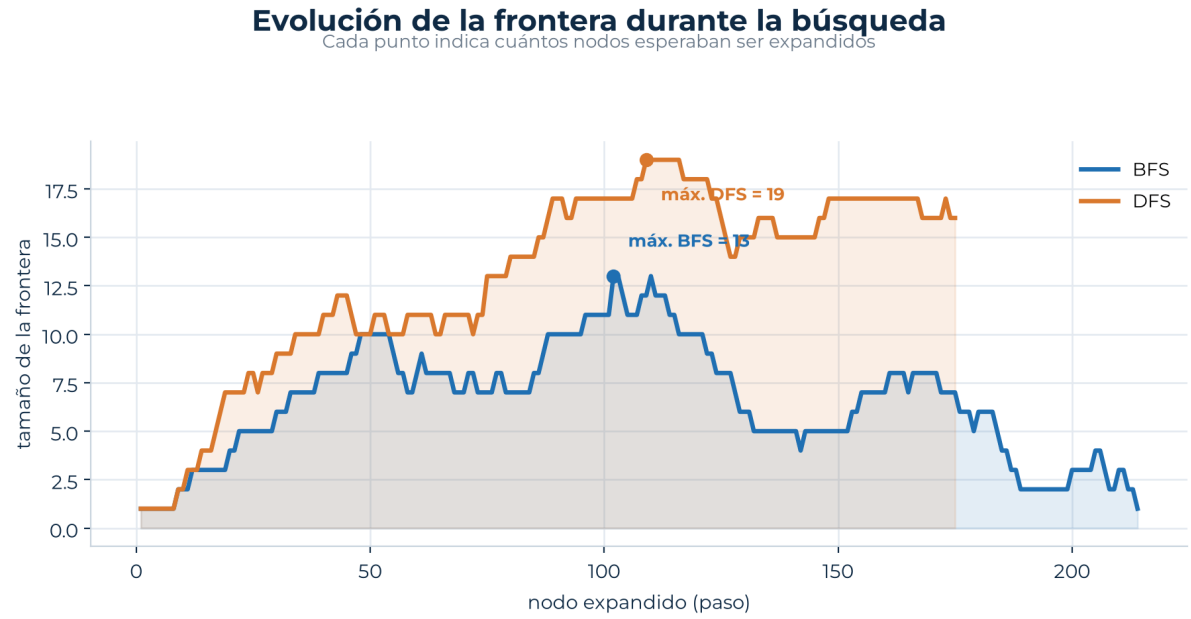


Figura 8. *Evolución del tamaño de la frontera durante la búsqueda.*

7. Análisis comparativo

7.1. Nodos expandidos y calidad del camino

DFS expandió 175 nodos frente a los 214 de BFS, es decir, un 18.2% menos. También evaluó menos aristas (382 frente a 467). Sin embargo, esa aparente ventaja tiene un coste alto: su camino es un 80.9% más largo. En otras palabras, DFS hace menos trabajo para encontrar una solución, pero la solución que encuentra es peor.

La explicación es estructural. BFS debe expandir todos los nodos con distancia menor que la de la meta antes de poder garantizar que no existe un camino más corto. Ese trabajo extra es el precio de la optimalidad. DFS, en cambio, se detiene en cuanto extrae la meta de la pila, aunque haya llegado a ella por un recorrido largo.

Tabla 3. Diferencias relativas entre BFS y DFS.

Métrica	BFS	DFS	Diferencia
Nodos expandidos	214	175	−18.2% (DFS)
Aristas evaluadas	467	382	−18.2% (DFS)
Longitud del camino	47	85	+80.9% (DFS)
Frontera máxima	13	19	+46.2% (DFS)
Tiempo medio (ms)	0.720	0.703	≈ 2.4%

7.2. Tiempo de ejecución

Los tiempos medidos son prácticamente iguales: 0.720 ms para BFS y 0.703 ms para DFS (diferencia ≈ 2.4%). Esto no significa que los algoritmos tengan el mismo coste teórico en todos los casos, sino que, en un grafo de solo 290 nodos, ambos son tan rápidos que el tiempo real está dominado por el ruido del sistema operativo y por detalles de implementación. Por eso, para comparar el trabajo realizado se usan métricas deterministas: nodos expandidos y aristas evaluadas.

En términos teóricos, ambos recorren cada nodo y cada arista como máximo una vez, por lo que su complejidad temporal es O(V + E). La diferencia práctica aparece en el número de nodos que cada estrategia necesita expandir antes de detenerse, y sobre todo en la calidad del camino obtenido.

7.3. Memoria y frontera

En esta instancia, la frontera máxima de BFS fue 13 nodos y la de DFS, 19. Es decir, DFS usó un 46.2% más de frontera, no menos. Este resultado recuerda que la idea de que «DFS siempre gasta menos memoria» es una simplificación: en búsqueda sobre grafos con conjunto de visitados, ambos algoritmos almacenan O(V) información, y el tamaño concreto de la frontera depende de la forma del grafo y del orden de exploración.

La ventaja clásica de DFS en memoria aparece en la búsqueda sobre árboles o en implementaciones recursivas con retroceso, donde la pila solo contiene el camino actual y su tamaño es O(b·m). En un laberinto con bucles y con un conjunto de visitados global, esa ventaja se diluye.

> **La paradoja de DFS**
>
> DFS expande menos nodos y evalúa menos aristas, pero devuelve un camino casi el doble de largo. BFS paga un poco más de trabajo para garantizar que la ruta sea la más corta. En un problema de navegación, esa garantía suele valer más que el ahorro de unas pocas expansiones.

8. Complejidad e idoneidad

8.1. Complejidad temporal y espacial

La Tabla 4 resume la complejidad de ambos algoritmos. V es el número de nodos (celdas transitables), E el número de aristas, b el factor de ramificación y m la profundidad máxima del árbol de búsqueda.

Tabla 4. Complejidad de BFS y DFS.

Algoritmo	Tiempo	Espacio	¿Completo?	¿Óptimo?
BFS	O(V + E)	O(V)	Sí	Sí (no ponderado)
DFS	O(V + E)	O(V) en grafos; O(b·m) en árboles	Sí (con visitados)	No

Ambos algoritmos son lineales en el tamaño del grafo porque, gracias al conjunto de visitados, cada nodo se expande una sola vez y cada arista se examina como máximo una vez. La diferencia está en la estructura de la frontera: una cola en BFS y una pila en DFS. Esa diferencia determina el orden de exploración y, con él, la optimalidad y el uso de memoria.

8.2. ¿Qué algoritmo conviene según el problema?

No existe un algoritmo «mejor» en abstracto: la elección depende de la pregunta que se hace al problema y de las restricciones de memoria y tiempo. La Tabla 5 resume los escenarios típicos.

Tabla 5. Idoneidad de BFS y DFS según el tipo de problema.

Problema	Algoritmo	Razón
Ruta más corta en laberinto o mapa sin pesos	BFS	Garantiza el camino con menos aristas.
Detección de ciclos, orden topológico, componentes conexas	DFS	Recorrido profundo y sencillo de implementar.
Backtracking (sudoku, N-reinas, planificación)	DFS	Explora decisiones y retrocede cuando falla.
Solo se necesita alguna ruta, no la más corta	DFS	Puede encontrar una solución con menos expansiones.
Grafo muy ancho y memoria limitada	DFS*	BFS puede almacenar una frontera enorme; DFS evita esa frontera.
Ruta mínima con costes distintos	Dijkstra / A*	BFS solo es óptimo si todas las aristas cuestan lo mismo.

** DFS evita la gran frontera de BFS, pero en búsqueda sobre grafos sigue almacenando el conjunto de visitados y la pila; la ventaja de memoria es más clara en árboles o en implementaciones recursivas con retroceso.*

Para el laberinto de esta actividad, BFS es la opción más adecuada: el objetivo es la ruta más corta, el grafo es pequeño (290 nodos) y la memoria no es una restricción. La frontera máxima de 13 nodos es perfectamente manejable. DFS solo sería preferible si bastara con cualquier ruta y se quisiera reducir el número de expansiones, pero el camino un 81% más largo es un mal negocio para un robot, un repartidor o cualquier aplicación de navegación.

Si el problema incluyera costes distintos por celda (por ejemplo, barro, agua o pendiente), BFS dejaría de ser óptimo y habría que usar Dijkstra. Si además se dispusiera de una heurística admisible, como la distancia Manhattan hasta la meta, A* encontraría la ruta óptima expandiendo menos nodos que BFS. Esa comparación queda como extensión natural de esta actividad.

9. Conclusiones

1. **Optimalidad:** ambos algoritmos resuelven el laberinto, pero solo BFS garantiza el camino más corto: 47 celdas frente a 85 de DFS.
2. **Eficiencia:** DFS expandió un 18.2% menos de nodos (175 frente a 214) y evaluó un 18.2% menos de aristas, pero pagó ese ahorro con un camino mucho más largo.
3. **Memoria:** en este laberinto, la frontera de DFS (19) fue mayor que la de BFS (13). La idea de que DFS siempre usa menos memoria no se sostiene en búsqueda sobre grafos con conjunto de visitados; el consumo depende de la instancia.
4. **Complejidad:** ambos tienen complejidad O(V + E) en tiempo y O(V) en espacio. Los tiempos medidos son prácticamente iguales (0.720 ms y 0.703 ms), así que a esta escala la decisión debe basarse en la optimalidad y en la forma de la frontera, no en la velocidad.
5. **Idoneidad:** para laberintos y rutas mínimas sin pesos, BFS es la elección correcta. DFS es preferible para exploración exhaustiva, detección de ciclos, orden topológico, backtracking y situaciones en las que basta con encontrar alguna solución.

En definitiva, la elección del algoritmo no depende de cuál es «mejor» en abstracto, sino de la pregunta que se hace al problema. Si la pregunta es «¿cuál es el camino más corto?», la respuesta es BFS. Si la pregunta es «¿existe algún camino?» o «¿qué estados son alcanzables?», DFS es una herramienta adecuada. Comprender esa diferencia es más valioso que memorizar una lista de ventajas y desventajas.

10. Referencias

[1] RUSSELL, S. y NORVIG, P. (2021). Artificial Intelligence: A Modern Approach, 4.ª ed. Pearson. Capítulo 3 (Solving Problems by Searching).

[2] CORMEN, T. H., LEISERSON, C. E., RIVEST, R. L. y STEIN, C. (2009). Introduction to Algorithms, 3.ª ed. MIT Press. Capítulo 22 (Elementary Graph Algorithms).

[3] KNUTH, D. E. (1997). The Art of Computer Programming, Vol. 1: Fundamental Algorithms, 3.ª ed. Addison-Wesley.

[4] SEDGEWICK, R. y WAYNE, K. (2011). Algorithms, 4.ª ed. Addison-Wesley. Capítulo 4 (Graphs).

[5] PYTHON SOFTWARE FOUNDATION (s. f.). Python Documentation. Disponible en: https://docs.python.org/3/

[6] HUNTER, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95. DOI: 10.1109/MCSE.2007.55.

[7] OPENCODE (2026). OpenCode: agente de inteligencia artificial para programación y análisis [software]. Disponible en: https://opencode.ai. Consultado el 25 de octubre de 2026.

Declaración de uso de inteligencia artificial

Para la elaboración de esta memoria se utilizó la herramienta de inteligencia artificial OpenCode [7], como apoyo en la implementación y depuración del código Python, la generación de las figuras y la maquetación de los archivos Word y PDF. La selección del problema, la definición del laberinto, la verificación de los resultados y la revisión del contenido corresponden a la persona estudiante, quien asume la responsabilidad sobre el material entregado. Todas las cifras reportadas provienen de ejecutar el código incluido en el anexo.

Anexo A — Código completo

A continuación se incluye el listado íntegro de los archivos de la carpeta codigo/. El código no requiere dependencias externas y puede ejecutarse directamente con python main.py.

A.1. laberinto.py — Laberinto, vecinos y representación del grafo

```
#!/usr/bin/env python3
"""
Laberinto de la Actividad 2.4.


Representación:
  '#' = muro
  '.' = pasillo transitable
  'S' = inicio
  'E' = salida


El laberinto se modela como un grafo no dirigido y no ponderado:
  - cada celda transitable es un nodo,
  - cada par de celdas adyacentes (arriba, abajo, izquierda, derecha)
    sin muro entre ellas es una arista.


Este módulo NO usa librerías externas: solo la biblioteca estándar.
"""
from __future__ import annotations


from typing import Iterator, List, Tuple


# Laberinto de 31 columnas x 19 filas (con bucles: hay varios caminos posibles)
LABERINTO: Tuple[str, ...] = (
    "###############################",
    "#S#.............#.#.........#",
    "#.#######.######..#.#.##..#.#.#",
    "#.........#.....#.#.......#.#.#",
    "######.########.#.#.#####.#.#.#",
    "#.............#.#.......#.#.#",
    "#.#.#######.###.#.#######.#.#.#",
    "#.#.#.......#.....#...#...#.#.#",
    "#.#....##.###.##.##.#.#.###...#",
    "#.#...#.#...#.#.....#.#...#.#.#",
    "#.#..##.###.#.#..####.###.#.#.#",
    "#.#.....#.....#.#.#...#...#.#.#",
    "#.#####...#####.#.#.###.###.###",
    "#.....#.....#.....#.#.....#...#",
    "#.###.#####.#.###.#.##########.#",
    "#...#.#...#.#.#...#.........#.#",
    "#.#.###.#...#.#.#.######.##.#.#",
    "#.#.....#.................E#",
    "###############################",
)


FILAS = len(LABERINTO)
COLUMNAS = len(LABERINTO[0])


Posicion = Tuple[int, int]



def buscar_simbolo(simbolo: str) -> Posicion:
    """Devuelve la posición (fila, columna) del símbolo indicado."""
    for f, fila in enumerate(LABERINTO):
        c = fila.find(simbolo)
        if c != -1:
            return (f, c)
    raise ValueError(f"No se encontró el símbolo {simbolo!r} en el laberinto")



INICIO: Posicion = buscar_simbolo("S")
META: Posicion = buscar_simbolo("E")



def es_transitable(pos: Posicion) -> bool:
    """Indica si una celda está dentro del laberinto y no es un muro."""
    f, c = pos
    return 0 <= f < FILAS and 0 <= c < COLUMNAS and LABERINTO[f][c] != "#"



def vecinos(pos: Posicion) -> Iterator[Posicion]:
    """Genera los vecinos transitables en orden: arriba, abajo, izquierda, derecha."""
    f, c = pos
    for df, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
        candidato = (f + df, c + dc)
        if es_transitable(candidato):
            yield candidato
```

```
def imprimir_laberinto(camino: List[Posicion] | None = None) -> str:
    """Devuelve el laberinto como texto, marcando el camino con '*'."""
    ocupadas = set(camino or [])
    lineas = []
    for f, fila in enumerate(LABERINTO):
        chars = []
        for c, ch in enumerate(fila):
            if (f, c) == INICIO:
                chars.append("S")
            elif (f, c) == META:
                chars.append("E")
            elif (f, c) in ocupadas and ch != "#":
                chars.append("*")
            else:
                chars.append(ch)
        lineas.append("".join(chars))
    return "\n".join(lineas)
```

A.2. busqueda_ciega.py — BFS, DFS y medición de resultados

```
#!/usr/bin/env python3
"""
Búsqueda ciega (no informada): BFS y DFS.


Ambos algoritmos reciben el laberinto como grafo implícito y devuelven:
  - el camino solución,
  - el número de nodos expandidos,
  - el número de aristas evaluadas,
  - el tamaño máximo de la frontera,
  - el tiempo de ejecución,
  - el orden de expansión (útil para las figuras).


Este módulo NO usa librerías externas: solo la biblioteca estándar.
"""
from __future__ import annotations


import time
from collections import deque
from dataclasses import dataclass, field
from typing import Callable, Dict, List, Optional


from laberinto import INICIO, LABERINTO, META, Posicion, vecinos



@dataclass
class Resultado:
    """Resultado de una ejecución de búsqueda."""


    algoritmo: str
    encontrado: bool
    camino: List[Posicion] = field(default_factory=list)
    expandidos: int = 0
    aristas_evaluadas: int = 0
    frontera_max: int = 0
    tiempo_seg: float = 0.0
    orden_expansion: List[Posicion] = field(default_factory=list)
    historial_frontera: List[int] = field(default_factory=list)


    @property
    def longitud_camino(self) -> int:
        """Número de celdas del camino (incluye inicio y meta)."""
        return len(self.camino)


    @property
    def coste(self) -> int:
        """Número de movimientos del camino (aristas recorridas)."""
        return max(0, len(self.camino) - 1)



def _reconstruir_camino(
    padre: Dict[Posicion, Optional[Posicion]], meta: Posicion
) -> List[Posicion]:
    """Reconstruye el camino desde el inicio hasta la meta usando el árbol de padres."""
    if meta not in padre:
        return []
    camino: List[Posicion] = []
    actual: Optional[Posicion] = meta
    while actual is not None:
        camino.append(actual)
        actual = padre[actual]
    camino.reverse()
    return camino



def bfs(
    inicio: Posicion = INICIO,
    meta: Posicion = META,
```

```
) -> Resultado:
    """
    Búsqueda en amplitud (Breadth-First Search).


    Explora por niveles con una cola FIFO. En un grafo no ponderado
    garantiza el camino de menor número de aristas (óptimo).
    Complejidad temporal: O(V + E).  Espacial: O(V).
    """
    t0 = time.perf_counter()
    padre: Dict[Posicion, Optional[Posicion]] = {inicio: None}
    frontera: deque[Posicion] = deque([inicio])
    orden: List[Posicion] = []
    expandidos = 0
    aristas_evaluadas = 0
    frontera_max = 1
    historial_frontera: List[int] = []


    while frontera:
        frontera_actual = len(frontera)
        historial_frontera.append(frontera_actual)
        frontera_max = max(frontera_max, frontera_actual)
        actual = frontera.popleft()
        expandidos += 1
        orden.append(actual)


        if actual == meta:
            break


        for vecino in vecinos(actual):
            aristas_evaluadas += 1
            if vecino not in padre:
                padre[vecino] = actual
                frontera.append(vecino)


    tiempo = time.perf_counter() - t0
    camino = _reconstruir_camino(padre, meta)
    return Resultado(
        algoritmo="BFS",
        encontrado=bool(camino),
        camino=camino,
        expandidos=expandidos,
        aristas_evaluadas=aristas_evaluadas,
        frontera_max=frontera_max,
        tiempo_seg=tiempo,
        orden_expansion=orden,
        historial_frontera=historial_frontera,
    )



def dfs(
    inicio: Posicion = INICIO,
    meta: Posicion = META,
) -> Resultado:
    """
    Búsqueda en profundidad (Depth-First Search).


    Explora por ramas con una pila LIFO. Encuentra *un* camino, pero no
    garantiza que sea el más corto.
    Complejidad temporal: O(V + E).  Espacial: O(V).
    """
    t0 = time.perf_counter()
    padre: Dict[Posicion, Optional[Posicion]] = {inicio: None}
    frontera: List[Posicion] = [inicio]
    orden: List[Posicion] = []
    expandidos = 0
    aristas_evaluadas = 0
    frontera_max = 1
    historial_frontera: List[int] = []


    while frontera:
        frontera_actual = len(frontera)
        historial_frontera.append(frontera_actual)
        frontera_max = max(frontera_max, frontera_actual)
        actual = frontera.pop()
        expandidos += 1
        orden.append(actual)


        if actual == meta:
            break


        # Se invierte el orden para que el recorrido visual vaya
        # arriba, abajo, izquierda, derecha.
        candidatos = list(vecinos(actual))
        aristas_evaluadas += len(candidatos)
        for vecino in reversed(candidatos):
            if vecino not in padre:
                padre[vecino] = actual
                frontera.append(vecino)


    tiempo = time.perf_counter() - t0
    camino = _reconstruir_camino(padre, meta)
    return Resultado(
        algoritmo="DFS",
        encontrado=bool(camino),
        camino=camino,
```

```
        expandidos=expandidos,
        aristas_evaluadas=aristas_evaluadas,
        frontera_max=frontera_max,
        tiempo_seg=tiempo,
        orden_expansion=orden,
        historial_frontera=historial_frontera,
    )



def medir_tiempo_promedio(
    funcion: Callable[[], Resultado], repeticiones: int = 200, lotes: int = 5
) -> float:
    """
    Ejecuta la búsqueda varias veces y devuelve el mejor tiempo medio por
    ejecución (en segundos).

    Se descarta una primera ejecución de calentamiento y se toman varios
    lotes, quedándose con el más rápido. Así se reduce el ruido del sistema
    operativo y la comparación entre algoritmos es más justa.
    """
    funcion()  # calentamiento (cachés de CPU, importaciones, etc.)
    mejor = float("inf")
    for _ in range(lotes):
        t0 = time.perf_counter()
        for _ in range(repeticiones):
            funcion()
        mejor = min(mejor, (time.perf_counter() - t0) / repeticiones)
    return mejor
```

A.3. main.py — Ejecución, comparación y guardado del informe

```
#!/usr/bin/env python3
"""
Actividad 2.4 — Algoritmos de búsqueda ciega (BFS y DFS).

Resuelve un laberinto de 31 x 19 celdas con búsqueda en amplitud (BFS) y
búsqueda en profundidad (DFS), y compara:
  - longitud del camino solución,
  - nodos expandidos,
  - aristas evaluadas,
  - tamaño máximo de la frontera,
  - tiempo de ejecución.

No requiere instalar ninguna librería: usa solo la biblioteca estándar.
"""
from __future__ import annotations

from pathlib import Path

from busqueda_ciega import bfs, dfs, medir_tiempo_promedio
from laberinto import COLUMNAS, FILAS, INICIO, LABERINTO, META, imprimir_laberinto

BASE_DIR = Path(__file__).resolve().parent
SALIDA = BASE_DIR.parent / "resultados_ejecucion_2.4.txt"
REPETICIONES = 200



def linea(caracter: str = "=", ancho: int = 68) -> str:
    return caracter * ancho



def main() -> None:
    print(linea())
    print("ACTIVIDAD 2.4 — BÚSQUEDA CIEGA: BFS Y DFS")
    print(linea())
    print(f"\nLaberinto: {FILAS} filas x {COLUMNAS} columnas "
          f"({FILAS * COLUMNAS} celdas, {sum(f.count('#') for f in LABERINTO)} muros)")
    print(f"Inicio: {INICIO}   Meta: {META}")
    print("\nRepresentación: grafo no dirigido y no ponderado")
    print("  - nodo  = celda transitable")
    print("  - arista = celdas adyacentes sin muro (arriba/abajo/izquierda/derecha)\n")

    print(linea("-"))
    print("LABERINTO")
    print(linea("-"))
    print(imprimir_laberinto())

    # Ejecuciones
    r_bfs = bfs()
    r_dfs = dfs()

    # Tiempo medio (más representativo que una sola ejecución)
    t_bfs = medir_tiempo_promedio(lambda: bfs(), REPETICIONES)
    t_dfs = medir_tiempo_promedio(lambda: dfs(), REPETICIONES)

    print("\n" + linea("-"))
```

```
    print("SOLUCIÓN CON BFS (camino más corto)")
    print(linea("-"))
    print(imprimir_laberinto(r_bfs.camino))


    print("\n" + linea("-"))
    print("SOLUCIÓN CON DFS (primer camino encontrado)")
    print(linea("-"))
    print(imprimir_laberinto(r_dfs.camino))


    print("\n" + linea("-"))
    print("COMPARACIÓN DE RESULTADOS")
    print(linea("-"))
    print(f"{'Métrica':<34}{'BFS':>14}{'DFS':>14}")
    print(f"{'-' * 62}")
    print(f"{'¿Encuentra solución?':<34}{'Sí' if r_bfs.encontrado else 'No':>14}"
          f"{'Sí' if r_dfs.encontrado else 'No':>14}")
    print(f"{'Celdas del camino':<34}{r_bfs.longitud_camino:>14}{r_dfs.longitud_camino:>14}")
    print(f"{'Movimientos (aristas)':<34}{r_bfs.coste:>14}{r_dfs.coste:>14}")
    print(f"{'Nodos expandidos':<34}{r_bfs.expandidos:>14}{r_dfs.expandidos:>14}")
    print(f"{'Aristas evaluadas':<34}{r_bfs.aristas_evaluadas:>14}{r_dfs.aristas_evaluadas:>14}")
    print(f"{'Frontera máxima':<34}{r_bfs.frontera_max:>14}{r_dfs.frontera_max:>14}")
    print(f"{'Tiempo medio (ms)':<34}{t_bfs * 1000:>14.4f}{t_dfs * 1000:>14.4f}")
    print(f"{'¿Camino óptimo?':<34}{'Sí':>14}{'No':>14}")
    print(f"\nTiempo medio calculado con {REPETICIONES} ejecuciones (mejor de 5 lotes).")


    print("\n" + linea("-"))
    print("COMPLEJIDAD")
    print(linea("-"))
    print("  BFS:  tiempo O(V + E)   ·   espacio O(V)")
    print("  DFS:  tiempo O(V + E)   ·   espacio O(V)")
    print("  V = celdas transitables, E = aristas del grafo del laberinto")


    print("\n" + linea("-"))
    print("CONCLUSIÓN")
    print(linea("-"))
    print(f"  BFS garantiza el camino más corto ({r_bfs.longitud_camino} celdas frente a")
    print(f"  {r_dfs.longitud_camino}), pero expande más nodos ({r_bfs.expandidos} frente a")
    print(f"  {r_dfs.expandidos}). En esta instancia DFS alcanzó una frontera algo mayor")
    print(f"  ({r_dfs.frontera_max} frente a {r_bfs.frontera_max}), porque apila todos los")
    print("  vecinos de cada nodo. Ambos son O(V + E) en tiempo y O(V) en espacio.")
    print("  Para laberintos con bucles conviene BFS si se busca la ruta mínima;")
    print("  DFS solo si basta con *alguna* ruta y se acepta que no sea óptima.")


    # Guardar informe
    with open(SALIDA, "w", encoding="utf-8") as f:
        f.write(linea() + "\n")
        f.write("ACTIVIDAD 2.4 — BÚSQUEDA CIEGA: BFS Y DFS\n")
        f.write(linea() + "\n\n")
        f.write(f"Laberinto: {FILAS} x {COLUMNAS} celdas\n")
        f.write(f"Inicio: {INICIO}   Meta: {META}\n")
        f.write("Representación: grafo no dirigido y no ponderado\n\n")
        f.write("Métrica;BFS;DFS\n")
        f.write(f"Encuentra solución;{'Sí' if r_bfs.encontrado else 'No'};"
                f"{'Sí' if r_dfs.encontrado else 'No'}\n")
        f.write(f"Celdas del camino;{r_bfs.longitud_camino};{r_dfs.longitud_camino}\n")
        f.write(f"Movimientos;{r_bfs.coste};{r_dfs.coste}\n")
        f.write(f"Nodos expandidos;{r_bfs.expandidos};{r_dfs.expandidos}\n")
        f.write(f"Aristas evaluadas;{r_bfs.aristas_evaluadas};{r_dfs.aristas_evaluadas}\n")
        f.write(f"Frontera máxima;{r_bfs.frontera_max};{r_dfs.frontera_max}\n")
        f.write(f"Tiempo medio (ms);{t_bfs * 1000:.4f};{t_dfs * 1000:.4f}\n")
        f.write(f"Camino óptimo;Sí;No\n\n")
        f.write("Camino BFS:\n")
        f.write(" -> ".join(str(p) for p in r_bfs.camino) + "\n\n")
        f.write("Camino DFS:\n")
        f.write(" -> ".join(str(p) for p in r_dfs.camino) + "\n\n")
        f.write("Laberinto con solución BFS:\n")
        f.write(imprimir_laberinto(r_bfs.camino) + "\n\n")
        f.write("Laberinto con solución DFS:\n")
        f.write(imprimir_laberinto(r_dfs.camino) + "\n")


    print(f"\nResultados guardados en: {SALIDA}")
    print("\n¡Terminado!")



if __name__ == "__main__":
    main()
```